ASSIGNMENT 6: DISASTER RELIEF RESOURCE ALLOCATION

Programme	T.Y. B.Tech	**Academic Year**	2026–27
Department	Information Technology	**Semester**	V
Course	Design Analysis and Algorithm Laboratory	**Course Code**	BIT25PC04
Name	Sarvam Tarapure	**PRN**	124B1F048

GitHub Repository: https://github.com/Black-sys-dot/Design-and-Analysis-of-Algorithms

Problem Statement

A major earthquake has affected a remote region. A relief team must transport essential supplies using a truck with a maximum capacity of W kilograms. There are N item types, each with a weight and a utility value that reflects its importance for meeting urgent needs. The aim is to select whole items whose total weight stays within the limit while maximizing the combined utility.

Tasks

- Model the decision using 0/1 Knapsack, where each item is either selected in full or not selected.
- Implement an algorithm that finds the highest-utility feasible selection.
- Compare Brute Force, Dynamic Programming and Greedy strategies.
- Consider priorities for medicines and perishable supplies, as well as multiple vehicles or changing conditions.

Course objective

Understand algorithmic complexity and select suitable algorithm-design strategies for practical problems.

Course outcomes

- Analyze algorithm performance using asymptotic notation.
- Obtain optimal solutions using Dynamic Programming.

Theory: 0/1 Knapsack Problem

Problem overview

The 0/1 Knapsack Problem asks which items should be taken from a collection when the carrying capacity is limited. Item i has weight weight[i] and value value[i]. Since an item cannot be split, the decision variable x[i] can only be 0 or 1.

The objective is to maximize the sum of item values while ensuring that the combined weight does not exceed W.

Mathematical model

Maximize: $\Sigma(value[i] \times x[i])$

Subject to: $\Sigma(weight[i] \times x[i]) \leq W$, with $x[i] \in \{0, 1\}$.

Brute Force approach

Brute Force generates all possible subsets, calculates the weight and value of each, ignores any subset that exceeds capacity, and retains the best feasible option. Its time complexity is O(2^N), so it becomes expensive as the number of items grows.

Dynamic Programming approach

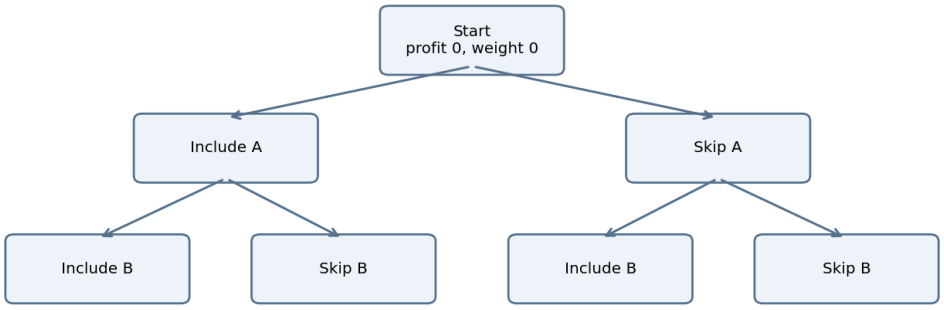
Dynamic Programming records solutions to smaller subproblems. Let dp[i][w] be the greatest value obtainable using the first i items with capacity w. When an item fits, compare the result from including it with the result from leaving it out.

Recurrence relation

Condition	DP update
weight[i–1] ≤ w	dp[i][w] = max(value[i–1] + dp[i–1][w – weight[i–1]], dp[i–1][w])
Otherwise	dp[i][w] = dp[i–1][w]

Time complexity: O(N × W) | Auxiliary space: O(N × W)

Include-or-skip decision tree



Each item creates two choices: take it or leave it.

Each item introduces two possible decisions: include it or skip it. The search tree illustrates why exhaustive enumeration grows quickly.

Greedy strategy and DP workflow

Greedy approach for Fractional Knapsack

- Sort items by their value-to-weight ratio, from highest to lowest.
- Take items in that order until the capacity is filled.

This greedy strategy is appropriate for Fractional Knapsack, where an item may be divided. It does not always give the optimal result for 0/1 Knapsack because items must be taken whole.

Inputs and outputs

Input: N items with weights w[i] and utility values v[i], plus truck capacity W.

Output: Maximum achievable utility and the list of selected item indices.

DP procedure

- Initialize dp[N+1][W+1] with zeros.
- For each item i from 1 to N, inspect every capacity w from 1 to W.
- If the item fits, store the larger of the include and exclude values; otherwise copy the previous row.
- The best utility is stored in dp[N][W].

Illustrative DP table

Illustrative DP table: best utility by capacity

Items used	0	1	2	3	4	5
0 items	0	0	0	0	0	0
A	0	1	1	1	1	1
A, B	0	1	7	8	8	8
A, B, C	0	1	7	11	12	18

Items: A = (weight 1, utility 1), B = (weight 2, utility 7), C = (weight 3, utility 11)

The table records the best utility for each item prefix and truck capacity. For these sample values, the best feasible selection at capacity 5 is B + C, with total weight 5 and utility 18.

Recovering the selected items

Once the DP table is complete, trace backward from dp[N][W] to find which items contribute to the best value.

- Start at the last row and capacity W.
- For i from N down to 1, compare dp[i][w] with dp[i–1][w].
- If the values differ, include item i and subtract its weight from w.
- Continue until all items have been checked or the remaining capacity reaches zero.

Example relief supplies



Utility values represent the importance assigned to each supply. The algorithm selects a combination that fits the truck, rather than simply choosing the item with the largest individual value.

Pseudocode: 0/1 Knapsack

```
function Knapsack(weight[], value[], N, W):
    create dp[N+1][W+1] and initialize all entries to 0

    for i from 1 to N:
        for w from 1 to W:
            if weight[i-1] <= w:
                dp[i][w] = max(
                    value[i-1] + dp[i-1][w-weight[i-1]],
                    dp[i-1][w])
            else:
                dp[i][w] = dp[i-1][w]

    maxUtility = dp[N][W]
    w = W
    includedItems = []

    for i from N down to 1:
        if dp[i][w] != dp[i-1][w]:
            includedItems.append(i)
            w = w - weight[i-1]

    return maxUtility, includedItems
```

Complexity and practical use

Method	Time	Explanation
Brute Force	O(2^N)	Examines every subset; expensive for large N.
Dynamic Programming	O(N × W)	Reuses stored solutions to smaller subproblems.
DP auxiliary space	O(N × W)	Stores results for each item-count and capacity pair.

Input/output summary

Input	Output
Item weights, utility values, item count N and capacity W.	Maximum total utility and selected item indices.

Possible extensions

- Multiple trucks require an extension that accounts for each vehicle's capacity.
- If the available inventory or capacity changes, the algorithm can be run again with updated inputs.
- Urgency can be reflected in the assigned utility values for medicines, food and other time-sensitive supplies.

Conclusion

The 0/1 Knapsack model supports systematic allocation of limited truck capacity among essential relief supplies. Dynamic Programming finds the maximum utility achievable within the weight limit, while traceback identifies the chosen items. This approach helps organize the loading of medicines, food and other supplies, and can be extended to changing inventory or multiple vehicles.